
abstract method

A method that *must* be overridden in every subclass.

Often, a base class cannot correctly implement an abstract method because it cannot know the detailed data it should operate on, or it may not be possible to implement a general algorithm. Instead, all subclasses need to implement their own version of the abstract method.

In our shapes example, we want the `Shape` class to be an abstract class so no client code can instantiate a `Shape` object. Further, our `Shape` class should indicate that all its subclasses need to implement the `clickedInside()`, `getArea()`, and `draw()` methods.

Python does not have a keyword to designate a class or method as abstract. However, the Python Standard Library contains the `abc` module, short for *abstract base class*, which is designed to help developers build abstract base classes and methods.

Let's take a look at what we need to do to build an abstract class with abstract methods. To begin, we need to import two things from the `abc` module:

```
from abc import ABC, abstractmethod
```

Next, we need to indicate that the class we want to act as an abstract base class should inherit from the `ABC` class, which we do by putting `ABC` inside parentheses after the class name:

```
class <classWeWantToDesignateAsAbstract>(ABC):
```

We then must use the special decorator `@abstractmethod` before any methods that must be overwritten by all subclasses:

```
@abstractmethod  
def <someMethodThatMustBeOverwritten>(self, ...):
```
